CS486 – Introduction to Databases

LAB
Topic 1: SQL data definition

Lecturer: Lê Thị Nhàn, PhD.
Lab instructors: Nguyễn Ngọc Toàn, MSc.
Nguyễn Ngọc Minh Châu, MSc.

1. Database operations

Create database

```
CREATE DATABASE [database name];
```

Example:

```
CREATE DATABASE TestDB;
```

SQL keywords are case insensitive

```
create database TestDB;
```

Remove database

```
DROP DATABASE [database name];
```

Example:

```
DROP DATABASE TestDB;
```

Select database

```
USE [database name];
```

Example:

```
USE TestDB;
```

Force remove database

```
-- Rollback all transactions
ALTER DATABASE Shop SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
-- Select master database
USE master;
-- Remove database
DROP DATABASE Shop;
```

2. Table operations

Create table

```
CREATE TABLE [table name]
(
[attribute 1] [data type 1] [constraint 1] [constraint 2]...,
[attribute 2] [data type 2] [constraint 1] [constraint 2]...,
...
PRIMARY KEY([list of attributes]),
UNIQUE([list of attributes]),
FOREIGN KEY([list of attributes]) REFERENCES [other table] ([list of attributes
in primary key])
);
```

Example:

```
CREATE TABLE Department (
    department_id VARCHAR(5),
    department_name VARCHAR(50) not null,
    office VARCHAR(5),
    department_head VARCHAR(9)
    -- PRIMARY KEY([list of columns]): One table SHOULD have one (and only one)
primary key but primary key may include one or more columns. The columns
in the primary key must not be null (NOT NULL). Primary key cannot be duplicated
    PRIMARY KEY (department_id)
);
CREATE TABLE Student (
    student_id VARCHAR(9) PRIMARY KEY,
    -- NOT NULL: Do not accept null value. Default is nullable
    student_name NVARCHAR(50) NOT NULL,
    -- CHECK ([condition]): Value must meet the condition (or combination of
conditions) in the clause
    gender CHAR(1) CHECK (gender IN ('F', 'M', 'O')),
    birthdate  DATETIME,
    class  VARCHAR(5),
```

```
    -- DEFAULT [value]: Value is used if a new record has no value for this
attribute
    department_id VARCHAR(5) NOT NULL DEFAULT 1,
    -- FOREIGN KEY: A foreign key must reference the primary key of another
table. The foreign key and the referenced primary key must have the same number
of columns, the same data types, and the same attribute order. Values in the
foreign key column(s) must either be NULL or match existing values in the
referenced table.
    FOREIGN KEY(department_id) REFERENCES Department(department_id)
)
```

Notes: Primary keys, unique constraints, and foreign keys can be defined directly next to a column if they involve only that single column.

Modify table definition

```
ALTER TABLE [table-name] ADD [attribute-name] [data type] [constraint 1]...
ALTER TABLE [table-name] DROP COLUMN [attribute] [constraint 1]...
ALTER TABLE [table-name] ALTER COLUMN [attribute] [constraint 1]...
ALTER TABLE [table-name] ADD CONSTRAINT...
ALTER TABLE [table-name] DROP CONSTRAINT...
```

Example:

```
ALTER TABLE Employee ADD CONSTRAINT FK_STUDENT_DEPARTMENT
FOREIGN KEY(department_id) REFERENCES Departments (department_id)


ALTER TABLE Employee DROP CONSTRAINT FK_STUDENT_DEPARTMENT
```

Composite primary key

A composite primary key is a primary key made up of two or more columns that together uniquely identify each row.

```
CREATE TABLE Enrollment (
    StudentID INT,
    CourseID INT,
    PRIMARY KEY (StudentID, CourseID)
);
```

Foreign Key Referencing Composite Primary Key

When referencing a composite primary key, the foreign key must include **all columns** in the **same order** and have **matching data types**.

```
CREATE TABLE Grade (
    StudentID INT,
    CourseID INT,
    GradeValue DECIMAL(3, 1),
    FOREIGN KEY (StudentID, CourseID) REFERENCES Enrollment(StudentID, CourseID)
);
```

Self-reference table

A column in a table can reference the primary key of the same table.

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(100),
    ManagerID INT,
    FOREIGN KEY (ManagerID) REFERENCES Employee(EmpID)
);
```

Circular Relationship

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(100),
    HeadID INT  );

CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    Name VARCHAR(100),
    DeptID INT,
    -- One employee works for one department (Employee -> Department)
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

```
-- Now add the foreign key from Department to Employee
ALTER TABLE Department
ADD FOREIGN KEY (HeadID) REFERENCES Employee(EmpID);
```

Remove table

```
DROP TABLE [table name];
```

Notes: In SQL, you cannot drop a table if it is being referenced by a foreign key from another table - unless you first remove or modify the reference constraint.

Data Insertion

```
INSERT INTO [table name] (column 1, column 2,...) VALUES (value 1, value 2);
```

Notes:

- When one table references another through a foreign key, you must insert data into the referenced table first, since the foreign key relies on existing values in that table.
- When self-reference exists, use NULL value for the foreign key column first then update the correct value for this column later.
- When circular relationships exist, use the NULL value for the foreign key in one table, then insert all data for the other table, finally update the value for the foreign key column for the first table.

Data update

```
UPDATE [table_name]
SET [column1] = [value1], [column2] = [value2], ...
WHERE [condition];
```

Delete

```
DELETE FROM [table name] WHERE [conditions];
```

Example:

```
DELETE FROM Departments WHERE department_id = 1;
```